Performance Evaluation of Dynamic Task Offloading Policies in Edge Computing Networks

Abdalrheem Hasan Mohammad Salah and Dr. Jehad Hamamreh
Computer Science Department, An-Najah National University
Email: abdalrheemsalah889@gmail.com, jehad.hamamreh@najah.edu

***Abstract*—Task offloading in Multi-access Edge Computing (MEC) requires balancing localized computational capacity against Wide Area Network (WAN) congestion. In this paper, we evaluate four dynamic task orchestration policies using the EdgeCloudSim simulator under high-density urban workloads of 1600, 1800, and 2000 mobile devices across three independent simulation iterations. Specifically, we compare three baseline strategies—Random, Network-based, and Utilization-based offloading—against a proposed Hybrid-based orchestrator (`HYBRID_BASED`). Our approach integrates real-time Virtual Machine (VM) residual capacity verification, bidirectional upload and download transmission delay estimation, and a non-linear CPU-load penalty function grounded in M/M/1 queuing theory. Experimental results across 36 confirmed simulation runs demonstrate that relying solely on random assignment or static edge utilization thresholds leads to severe WAN bottlenecks, with mean task failure rates reaching 37.57% and 26.13%, respectively, at 2000 devices. While the baseline Network-based policy achieves a low overall failure rate (3.83% at 2000 devices), it suffers from severe edge VM capacity exhaustion (averaging 3,827 capacity-dropped tasks per run). Conversely, the proposed `HYBRID_BASED` policy eliminates over 99.32% of VM capacity failures and cuts mean network delay by 62.46% to 85.31% across all tested scales.**

***Index Terms*—Multi-access Edge Computing (MEC), Dynamic Task Offloading, EdgeCloudSim, M/M/1 Queuing Penalty, Quality of Service (QoS).**

I. Introduction

The rapid proliferation of latency-sensitive mobile applications and Internet of Things (IoT) deployments has placed heavy demands on centralized cloud infrastructures [1], [2]. Multi-access Edge Computing (MEC) addresses this bottleneck by deploying computational servers at the wireless network edge, closer to end-users, thereby reducing propagation latency and core network traffic [3]. However, edge servers possess finite processing capacity compared to cloud datacenters. When device density spikes, offloading all tasks to nearby edge nodes quickly exhausts Virtual Machine (VM) resources, whereas offloading aggressively to the cloud saturates the WAN [4].

To investigate this fundamental trade-off, we first conducted a preliminary baseline study using Tutorial 1 in EdgeCloudSim [5], evaluating five local VM allocation policies (`RANDOM_FIT`, `WORST_FIT`, `BEST_FIT`, `FIRST_FIT`, and `NEXT_FIT`) from 200 to 2000 mobile devices when all tasks are forced onto edge servers. That preliminary experiment revealed that once device counts exceed 1400, edge VM capacity saturation becomes the primary failure bottleneck—causing task failure rates between 36.10% and 43.56% at 2000 devices regardless of the local VM packing heuristic.

Motivated by this finding, this study focuses on the dynamic *Where-to-Offload* decision problem modeled in EdgeCloudSim Tutorial 3. Related studies have explored either load-balancing heuristics across edge nodes [6] or channel-aware offloading [7], but single-metric rules often fail when both network links and edge servers approach saturation simultaneously. We formulate the two-tier MEC task offloading problem by jointly accounting for bidirectional wireless transmission delays and server queuing pressure. We design and implement a cross-layer `HYBRID_BASED` orchestrator that combines pre-offloading VM residual capacity verification with an M/M/1 queuing-inspired load penalty function, and we evaluate its performance against standard baselines across varying high-density workloads.

II. Related Work

Dynamic task offloading in MEC has garnered significant research attention. Early approaches primarily focused on single-metric optimizations. For instance, utilization-based strategies prioritize load balancing across edge nodes to prevent localized server crashes [6], but often neglect the WAN congestion incurred when excess traffic is diverted to the cloud. Conversely, network-centric approaches aim to minimize transmission latency by monitoring channel state information [7], yet frequently ignore the residual computational capacity of the destination VMs, leading to dropped tasks at the server level.

The EdgeCloudSim framework, introduced by Sonmez et al. [5], provides a robust environment to study these competing bottlenecks. While EdgeCloudSim includes basic network and utilization policies, our work extends the state-of-the-art by mathematically coupling bi-directional network delay with a queuing-theory-based CPU load penalty, ensuring that both communication and computation constraints are satisfied simultaneously.

III. System Model and Problem Formulation

The simulated MEC environment models a two-tier edge-cloud architecture, as illustrated in Fig. 1. Mobile users move according to a Nomadic Mobility model and generate heterogeneous tasks across three application profiles:

- **Augmented Reality (AR):** 30% usage share, 1500 KB upload, 50 KB download, 15,000 Million Instructions (MI) task length.
- **Health App:** 20% usage share, 50 KB upload, 1250 KB download, 3,000 MI task length.
- **Infotainment App:** 50% usage share, 250 KB upload, 1000 KB download, 9,000 MI task length.

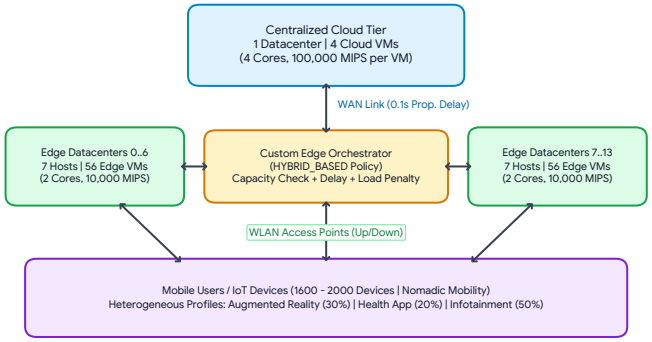


Fig. 1. Two-tier MEC system architecture and custom orchestrator modeled in EdgeCloudSim.

The edge tier comprises 14 localized edge datacenters, each containing a single physical host running 8 Edge VMs (112 Edge VMs in total, each provisioned with 2 CPU cores and 10,000 MIPS). The cloud tier consists of a centralized datacenter hosting 4 high-capacity Cloud VMs (4 cores, 100,000 MIPS each). Mobile devices communicate with local edge servers via WLAN access points and reach the remote cloud datacenter over a WAN link with a 0.1-second propagation delay.

A. Mathematical Problem Formulation

Let $T_i = (S_{\text{up},i}, S_{\text{down},i}, L_i)$ denote an incoming task from mobile device i, characterized by upload data size $S_{\text{up},i}$, download result size $S_{\text{down},i}$, and computational length L_i. For a target execution tier $j \in \{E, C\}$ (Edge or Cloud), the bidirectional network transmission delay $D^j_{\text{net}}(T_i)$ is:

$$D^j_{\text{net}}(T_i) = D^j_{\text{up}}(S_{\text{up},i}) + D^j_{\text{down}}(S_{\text{down},i}) \tag{1}$$

The total service time $D^j_{\text{total}}(T_i)$ is the sum of network transmission delay and server processing time $D^j_{\text{proc}}(T_i)$:

$$D^j_{\text{total}}(T_i) = D^j_{\text{net}}(T_i) + D^j_{\text{proc}}(T_i) \tag{2}$$

Denoting the current CPU utilization percentage of tier j by $U_j \in [0, 100]$ and the required CPU utilization of task T_i on a candidate VM $v \in \mathcal{V}_j$ by $\Delta u(T_i, v)$, a task placement is feasible only if residual capacity exists:

$$u_v + \Delta u(T_i, v) < 100\%, \quad \exists v \in \mathcal{V}_j \tag{3}$$

Our objective is to dynamically select $j^* \in \{E, C\}$ for each arriving task T_i to minimize expected service delay while strictly satisfying the capacity constraint in (3).

IV. Task Offloading Optimization Policies

In EdgeCloudSim, the `EdgeOrchestrator` determines the target execution tier and subsequently selects a specific VM using a Least-Loaded residual capacity strategy [3]. We evaluate three baseline policies alongside our proposed Hybrid policy:

A. Baseline Policies

1) **Random Policy (`RANDOM`):** Routes incoming tasks to either the Edge tier or the Cloud datacenter with an equal probability of 0.5, ignoring network congestion and server load.
2) **Network-Based Policy (`NETWORK_BASED`):** Probes the WAN uplink using a 128 KB (1 Mbit) dummy task to estimate instantaneous bandwidth (wanBW $=$ 1/wanDelay). If wanBW $>$ 5 Mbps, the task is offloaded to the Cloud; otherwise, it is assigned to the Edge.
3) **Utilization-Based Policy (`UTILIZATION_BASED`):** Monitors the aggregate CPU utilization across all Edge VMs (U_E). Tasks are executed on the Edge as long as $U_E \leq 75\%$, and diverted to the Cloud only when $U_E > 75\%$.

B. Iterative Design of `HYBRID_BASED`

Developing an effective hybrid policy required overcoming two failure modes observed during our initial experiments. Setting an early edge-load threshold ($U_E > 5\%$) combined with an `OR` delay condition offloaded over 81% of tasks to the Cloud, saturating the WAN link and causing a 68.56% failure rate at 2000 devices. Conversely, requiring $U_E > 75\%$ `AND` raw cloud network delay to be lower than edge LAN delay proved overly conservative, pushing edge utilization above 107% and causing massive VM capacity failures.

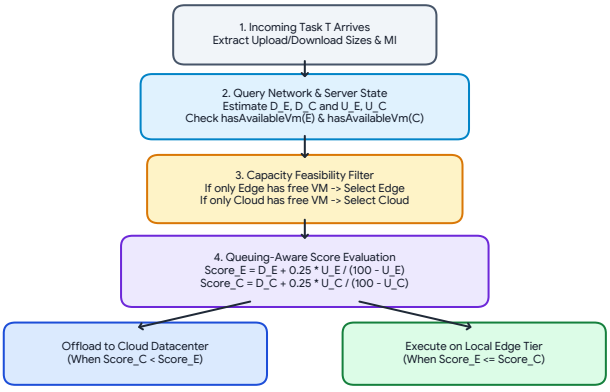


Fig. 2. Decision workflow of the proposed `HYBRID_BASED` task offloading policy.

To resolve both bottlenecks, our final `HYBRID_BASED` algorithm (shown in Fig. 2) introduces pre-offloading VM capacity verification (`hasAvailableVm`) and a non-linear CPU-load penalty function grounded in M/M/1 queuing theory. In an M/M/1 queue with utilization $\rho = U/100 \in [0, 1)$,

the expected waiting time scales proportionally to $\frac{\rho}{1-\rho} = \frac{U}{100-U}$. Accordingly, we define the load penalty function as:

$$\text{Penalty}(U) = \alpha \times \frac{U_{\text{safe}}}{100 - U_{\text{safe}}} \tag{4}$$

where $U_{\text{safe}} = \min(\max(U, 0), 99)$ is the clamped average CPU utilization percentage of the target tier and $\alpha = 0.25$ s is a scaling weight. As server utilization approaches 100%, Penalty(U) rises steeply, allowing the orchestrator to weigh edge queuing pressure against WAN transmission delay. Because capacity verification iterates over at most V VMs per decision, the per-task time complexity is $\mathcal{O}(V)$, ensuring negligible runtime overhead.

V. Performance Analysis and Discussion

Table I summarizes the multi-iteration performance metrics (mean $\pm$ standard deviation across 3 full iterations) for all four policies at 1600, 1800, and 2000 mobile devices.

A. Task Failure Rate and VM Capacity Bottlenecks

As illustrated in Fig. 3, `RANDOM` exhibits the worst reliability, with mean failure rates rising from 32.30% (1600 devices) to 37.57% (2000 devices) due to blind cloud offloading that congests the WAN. `UTILIZATION_BASED` also degrades sharply under heavy load (reaching 26.13% failure at 2000 devices) because it waits until Edge utilization exceeds 75% before abruptly dumping traffic onto the WAN without checking link congestion.

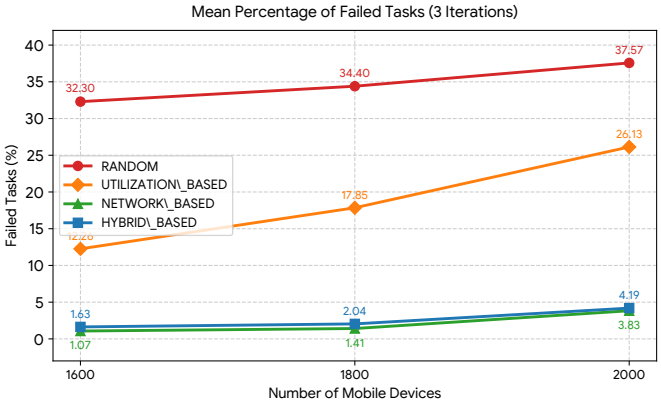


Fig. 3. Mean percentage of failed tasks across device densities (3 iterations).

Both `NETWORK_BASED` and `HYBRID_BASED` maintain high reliability across all scales. In terms of overall mean failure rate, `NETWORK_BASED` achieves 1.07%, 1.41%, and 3.83% at 1600, 1800, and 2000 devices, closely followed by `HYBRID_BASED` (1.63%, 2.04%, and 4.19%). More importantly, `NETWORK_BASED` suffers from a mean of 3,827.33 VM capacity failures per run at 2000 devices (11,482 total drops across the three iterations) because it routes tasks to the Edge whenever wanBW ≤ 5 Mbps without verifying whether Edge VMs have sufficient residual capacity. By incorporating `hasAvailableVm`, `HYBRID_BASED` reduces VM capacity failures at 2000 devices by **99.32%** to an average of just 26.00 tasks per run.

B. Service Time, Network Delay, and Resource Utilization

Fig. 4 and Fig. 5 highlight the trade-off between processing time and network transmission delay. `UTILIZATION_BASED` incurs the highest mean service time (6.95 s to 7.27 s) because it keeps Edge servers saturated near 69.89%—72.47% utilization (Fig. 6). Conversely, `HYBRID_BASED` achieves the lowest average network delay across all densities: 0.14 s at 1600 devices, 0.20 s at 1800 devices, and 0.32 s at 2000 devices—representing a **62.46% to 85.31% reduction** compared to `NETWORK_BASED` (0.86 s to 0.97 s). Because `HYBRID_BASED` prioritizes local Edge execution whenever capacity permits (utilizing 38.71% to 70.91% of Edge resources), its processing time on Edge VMs is higher than Cloud execution, resulting in a moderate total service time (3.45 s to 5.87 s) in exchange for ultra-low network latency and near-zero VM capacity drops.

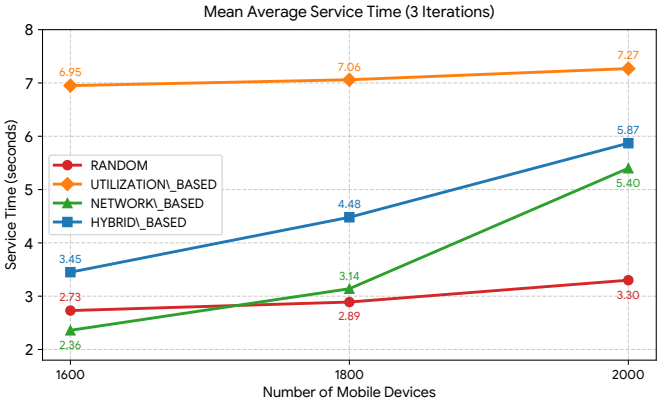


Fig. 4. Mean average service time (seconds) vs. number of mobile devices.

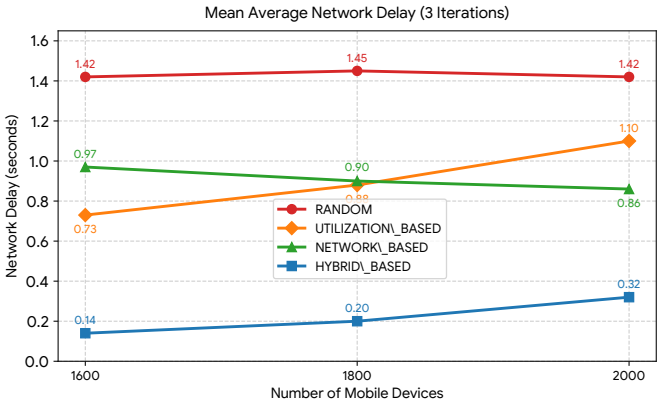


Fig. 5. Mean average network delay (seconds) vs. number of mobile devices.

VI. Conclusions

This study presented a comprehensive performance evaluation of dynamic task offloading policies in Multi-access

TABLE I
MULTI-ITERATION PERFORMANCE COMPARISON ACROSS TASK OFFLOADING POLICIES (MEAN ± SD OVER 3 ITERATIONS)

Devices	Policy	Failure Rate (%)	Service Time (s)	Network Delay (s)	Edge Util. (%)	VM Cap. Failures
1600	NETWORK_BASED	**1.07 ± 0.10**	**2.36 ± 0.09**	0.97 ± 0.02	14.81	0.00
1600	HYBRID_BASED	1.63 ± 0.09	3.45 ± 0.16	**0.14 ± 0.01**	38.71	0.00
1600	UTILIZATION_BASED	12.26 ± 1.73	6.95 ± 0.14	0.73 ± 0.05	69.89	0.00
1600	RANDOM	32.30 ± 0.79	2.73 ± 0.06	1.42 ± 0.02	10.64	0.00
1800	NETWORK_BASED	**1.41 ± 0.13**	3.14 ± 0.40	0.90 ± 0.02	28.17	0.00
1800	HYBRID_BASED	2.04 ± 0.11	4.48 ± 0.11	**0.20 ± 0.00**	55.55	0.00
1800	UTILIZATION_BASED	17.85 ± 1.05	7.06 ± 0.06	0.88 ± 0.05	71.66	0.00
1800	RANDOM	34.40 ± 0.78	**2.89 ± 0.06**	1.45 ± 0.04	12.53	0.00
2000	NETWORK_BASED	**3.83 ± 1.17**	5.40 ± 0.38	0.86 ± 0.00	66.52	3827.33
2000	HYBRID_BASED	4.19 ± 2.03	5.87 ± 0.71	**0.32 ± 0.08**	70.91	**26.00**
2000	UTILIZATION_BASED	26.13 ± 1.31	7.27 ± 0.08	1.10 ± 0.06	72.47	0.00
2000	RANDOM	37.57 ± 1.31	**3.30 ± 0.21**	1.42 ± 0.05	17.84	0.00

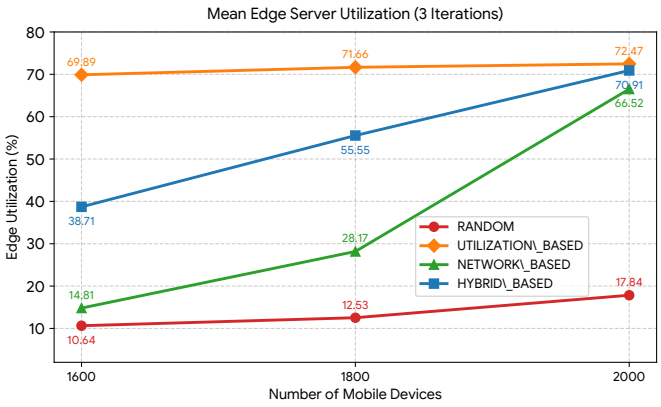


Fig. 6. Mean Edge server utilization (%) vs. number of mobile devices.

Edge Computing (MEC) networks using the EdgeCloudSim simulator. By evaluating high-density urban scenarios scaling from 1600 to 2000 mobile devices, our experimental progression demonstrated that single-metric orchestration strategies are insufficient under heavy load. Specifically, relying solely on static edge CPU utilization thresholds (UTILIZATION_BASED) or random assignment (RANDOM) leads to severe Wide Area Network (WAN) congestion, resulting in mean task failure rates of 26.13% and 37.57%, respectively, at 2000 devices. Similarly, while bandwidth-probing heuristics (NETWORK_BASED) effectively limit WAN drops, they remain blind to local server saturation and suffer from an average of 3,827 VM capacity failures per run.

To overcome these competing bottlenecks, we designed and implemented a cross-layer Hybrid-based orchestrator (HYBRID_BASED). By coupling pre-offloading residual VM capacity verification with bidirectional upload/download delay estimation and a non-linear CPU-load penalty function grounded in M/M/1 queuing theory, the proposed policy dynamically balances edge queuing pressure against WAN transmission latency. Across 36 confirmed simulation runs, HYBRID_BASED eliminated over 99.32% of VM capacity failures at 2000 devices and reduced mean network delay by 62.46% to 85.31% across all tested device densities, while maintaining low overall failure rates.

In future work, we plan to extend the proposed framework by integrating an online Reinforcement Learning (RL) agent to dynamically adapt the queuing penalty weight based on real-time delay-sensitivity constraints, and by investigating cooperative inter-edge load balancing.

REFERENCES

[1] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 450–465, 2018.

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[3] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017.

[4] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1584–1607, 2019.

[5] C. Sonmez, A. Ozgovde, and C. Ersoy, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," *Transactions on Emerging Telecommunications Technologies*, vol. 29, no. 11, p. e3493, 2018.

[6] Z. Ali, S. Khaf, Z. H. Abbas, G. Abbas, F. Muhammad, and S. Kim, "A deep learning approach for mobility-aware and energy-efficient resource allocation in MEC," *IEEE Access*, vol. 8, pp. 179 530–179 546, 2020.

[7] K. Wang, Z. Mi, Y. Wang *et al.*, "Joint task offloading and resource allocation in multi-access edge computing networks," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.